EXAMEN PRÁCTICO 02

Sistema de Control Serial Inteligente

UART • PWM • ADC • Entrada digital • Procesamiento Dual-Core

Equipo / alumno:		**Fecha:**	23/10/2026
Integrantes:	Camila Rodríguez Rosas Hamid Susen Santiago García Jonathan Hernández Lazcano	**Hora de inicio:**	5:00 pm

1. Propósito del examen

Construir y demostrar un único sistema embebido que integre los temas principales vistos en las sesiones recientes. La Raspberry Pi Pico deberá recibir comandos, controlar salidas, leer entradas y mantener una tarea de adquisición en segundo plano sin detener la comunicación.

Duración total: 2 horas. Este tiempo incluye montaje, programación, pruebas, correcciones y la demostración completa frente al docente. Pueden trabajar de manera individual o en equipo. Si trabajan en equipo, deben registrar los nombres completos de todos los integrantes. Durante la evaluación y la demostración deberán mantener la cámara encendida.

2. Material y montaje mínimo

- Raspberry Pi Pico / Pico W / Pico 2 / Pico 2 W con MicroPython.
- Computadora con Thonny y cable USB de datos.
- 2 LEDs con resistencias (o shield de LEDs equivalente).
- 1 botón/pulsador.
- 1 fuente analógica de sesiones anteriores: potenciómetro, LDR, NTC u otro sensor analógico aprobado por el docente.
- Jumpers y protoboard, si el shield no incluye los elementos necesarios.

Asignación recomendada de pines

Elemento	Pin sugerido	Función
LED digital	GP13	Encendido/apagado
LED PWM	GP15	Brillo 0–100 %
Botón	GP16	Entrada con PULL_UP
ADC	GP26 / ADC0	Sensor o potenciómetro 0–3.3 V

SEGURIDAD: una entrada ADC de la Pico no debe recibir más de 3.3 V.

3. Arquitectura obligatoria

CORE 0	CORE 1
• Recibe y procesa comandos seriales • Controla LED digital y PWM • Genera ACK / ERR / STATUS	• Lee el ADC periódicamente • Actualiza el valor/porcentaje ADC • Incrementa sample_count • No debe detenerse mientras Core 0 espera comandos

Prueba de concurrencia obligatoria: al ejecutar STATUS dos veces con una pausa entre ambas, sample_count deberá aumentar aunque no se hayan enviado otros comandos.

4. Protocolo de comandos obligatorio

El sistema deberá reconocer exactamente estas cinco familias de comandos:

Comando	Acción	Respuesta esperada	Tipo
PING	Comprueba comunicación.	PONG	Simple

Comando	Acción	Respuesta esperada	Tipo
LED,0 / LED,1	Apaga o enciende el LED digital.	ACK,LED,0/1	Escritura
PWM,n	Ajusta brillo del LED PWM. n = 0...100.	ACK,PWM,n	Escritura + PWM
STATUS	Reporta el estado completo del sistema.	STATUS,LED=...,PWM=...,ADC=...,BTN=...,SAMPLES=...	Lectura
ALL,0 / ALL,1	Comando compuesto: cambia LED digital y PWM juntos.	ACK,ALL,0/1	Compuesto

5. Requisitos funcionales

- El porcentaje PWM debe convertirse correctamente al rango de duty_u16() usando 0–65535.
- El ADC debe leerse en Core 1 y convertirse al menos a porcentaje 0–100 %.
- STATUS debe reflejar valores reales actuales, no valores escritos manualmente.
- El botón debe poder leerse y su estado debe aparecer en STATUS (BTN=0/1).
- Al presionar el botón, el sistema debe producir un reporte de estado o un evento visible. Se acepta una solución mediante segundo núcleo, interrupción o estrategia equivalente.
- El programa debe continuar funcionando después de un comando inválido.
- No se permite reiniciar la placa entre pruebas normales.

6. Manejo de errores obligatorio

Como mínimo deben detectarse los siguientes casos:

Entrada de prueba	Problema	Respuesta válida
PWM,150	Fuera del rango permitido	ERR,PWM_RANGE
LED,7	Valor inválido	ERR,BAD_VALUE
PWM	Trama incompleta	ERR,BAD_FORMAT
HOLA	Comando desconocido	ERR,UNKNOWN_COMMAND

7. Secuencia de demostración frente al docente

La demostración forma parte de las mismas 2 horas del examen; no habrá tiempo adicional después. El equipo completo (o el alumno si trabaja individualmente) debe estar presente, con cámara encendida, y mostrar el sistema funcionando en vivo.

1. Ejecutar el programa desde cero sin errores.
2. Enviar PING y mostrar PONG.
3. Enviar LED,1 y LED,0; comprobar físicamente el LED.
4. Enviar PWM,25; PWM,50; PWM,100 y mostrar el cambio de brillo.
5. Mover el potenciómetro/sensor y ejecutar STATUS; demostrar que ADC cambia.
6. Presionar el botón y demostrar su lectura/evento de estado.
7. Enviar ALL,1 y ALL,0; demostrar que actúan sobre más de una salida.
8. Ejecutar STATUS dos veces; comprobar que SAMPLES aumentó (prueba Dual-Core).
9. Probar al menos dos errores de la tabla anterior y demostrar que el programa sigue funcionando.

La demostración es acumulativa y debe completarse dentro de las 2 horas. El sistema debe permanecer ejecutándose durante todas las pruebas. Si una función falla, puede corregirse mientras quede tiempo. La última demostración realizada dentro del periodo del examen será la que cuente.

8. Distribución de las 2 horas del examen

Tiempo	Etapa	Meta
0–10 min	Lectura + montaje	Registrar integrantes y verificar LEDs, botón, ADC y conexión USB.
10–35 min	Arquitectura base	Core 1 ADC + variables compartidas + Core 0.
35–70 min	Parser serial	Implementar los cinco comandos.
70–90 min	PWM + STATUS + botón	Integrar entradas/salidas reales.
90–105 min	Errores y pruebas	Validar entradas incorrectas y robustez.

Tiempo	Etapa	Meta
105–120 min	Demostración final	Mostrar TODOS los casos frente al docente, con cámara encendida.

9. Evaluación (100 puntos)

Criterio	Puntos	Evidencia requerida	Puntaje
Montaje y arranque físico	10	Pico ejecuta sin errores; LEDs, botón y ADC correctamente conectados.	
5 comandos seriales	25	PING, LED, PWM, STATUS y ALL funcionan según especificación.	
PWM	10	Control visible 0–100 % y conversión correcta a duty_u16.	
ADC	10	Lectura real cambia; conversión a porcentaje incluida en STATUS.	
Dual-Core	15	Core 1 adquiere ADC y SAMPLES continúa aumentando.	
Botón / entrada digital	10	Lectura correcta y reporte/evento de estado.	
Errores y robustez	10	Detecta entradas inválidas sin detener el sistema.	
Demostración y explicación	10	Explica flujo TX/RX, PWM, ADC y división Core 0/Core 1.	

10. Condiciones de entrega

• Modalidad: individual o por equipo. Si es por equipo, escriban los nombres completos de todos los integrantes.

• Cámara: deberá permanecer encendida durante la evaluación y especialmente durante la demostración en vivo.

- Mostrar el sistema funcionando en vivo antes de terminar la sesión.
- Entregar el archivo main.py utilizado en la demostración.
- Subir al repositorio de GitHub del curso una carpeta identificable para este examen.
- Incluir un README breve con: integrantes, diagrama/pines usados, lista de comandos y una captura/foto del sistema.
- Cada integrante debe poder explicar qué hace al menos una parte del programa.

11. Restricciones

- No se acepta una demostración basada únicamente en prints simulados: las salidas deben cambiar físicamente.
- No se permite sustituir el ADC por un número fijo.
- No se permite eliminar el manejo de errores para simplificar el parser.
- El código puede reutilizar estructuras vistas en clase, pero el equipo debe comprenderlas y poder modificarlas durante la evaluación.

BONUS (+5): implementar un comando adicional útil (por ejemplo HELP, TOGGLE, PWM+/- o una alarma por ADC) y explicarlo correctamente.

12. Checklist rápido antes de llamar al docente

☐ PING → PONG
☐ LED,1 / LED,0
☐ PWM,25 / PWM,50 / PWM,100
☐ STATUS muestra LED, PWM, ADC, BTN y SAMPLES
☐ ALL,1 / ALL,0
☐ El ADC cambia al mover el sensor
☐ El botón se detecta
☐ SAMPLES continúa aumentando
☐ PWM,150 produce ERR
☐ HOLA produce ERR y el programa continúa